Project 3 – Assess Learners

By: Nicholas Daponte
ndaponte3@gatech.edu

IMPORTANT NOTE:

Resubmission from Summer 2024(Course Retake)

***Abstract*—**Three experiments were conducted and simulated in Python on the financial stock market Istanbul dataset. Decision Tree Learner(DTLearner), Random Tree Learner(RTLearner), and Boostrap Aggregating Learner(BagLearner) regression models were used to predict stock returns for the Emerging Markets index(EM). Overfitting was slightly reduced but not eliminated in bagging and DTLearner indicated greater performance than RTLearner for Mean Absolute Logarithmic Error(MALE) and Median Absolute Percentage Error(MdAPE) across all leaf sizes.

1 INTRODUCTION

The classical Decision Tree Learner (DTLearner) provides a clear and interpretable model structure, making it valuable for understanding feature importance in regression tasks. The Random Tree Learner (RTLearner) introduces randomness to reduce overfitting and enhance model generalization through random subset of features at each split. The Bootstrap Aggregating Learner (BagLearner) and the Insane Learner further improve predictive accuracy and robustness by combining multiple learners, thereby leveraging ensemble learning techniques to handle the complexities of stock market data more effectively. The purpose of this report is to conduct three experiments on the Istanbul raw dataset that contains X values consisting of stock returns and a Y label of the Emerging Markets(EM) index stock return(the prediction label). The overarching goal is to compare Regression Tree algorithms on this financial dataset to evaluate and compare performance of the multiple models.

2 METHODS

All tests will be performed using the testlearner.py script containing implementations for 3 learners: DTLearner(decision tree), RTLearner(random tree), and

BagLearner(Booststrap) algorithms. Executing the tests use the command PYTHONPATH = ../:. Python testlearner.py Data/Istanbul.csv. The add_evidence() method uses the train data and the build_tree() method creates the tree by training the model with the JR Quinlan algorithm(Quinlan, 1986). The query() method produces predictions using the constructed tree and input data for the learners are provided as NumPy nd-arrays where the output generated is a 1D array of the predicted stock returns for the EM index. The Istanbul.csv file is used for all 3 experiments as the raw dataset and is treated as non-time series , thus the random train/split of the data can be incorporated without consideration of temporal data leakage. All 3 experiments use the same train/split of 60/40 ratio with the input features being the returns of the other stocks aside from the EM single label. The NumPy random seed API is utilized to fix the same random split for fair comparison of each experiment with the raw dataset.

3 EXPERIMENT 1

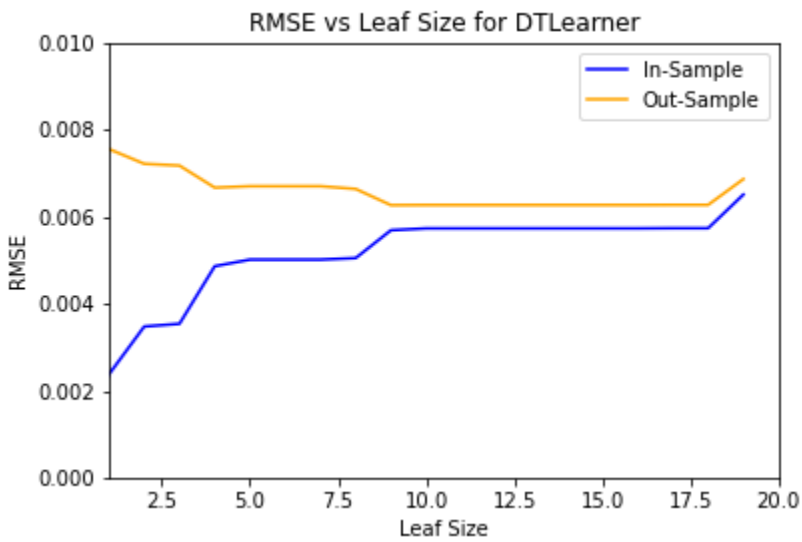

Figure 1. RMSE versus Leaf Size of In-Sample and Out-of-sample for the DTLearner on the Istanbul.csv raw data file with random shuffle 60/40 Train/Test data split.

Does overfitting occur with respect to leaf size and for which values does overfitting occur?

Overfitting occurs initially at a leaf size less than 9 with a steady plateau in RMSE at a leaf size of 8 where increasing Leaf Size does not change performance and overfitting decreases as the model becomes simpler to the point of underfitting resulting in a rising RMSE in both in and out-of-sample. The firection of overfitting is to the left or with decreasing Leaf Size.

Overfitting occurs when a model captures not only the true patterns in the training data but also the noise and outliers, which results in poor performance on new, unseen data. In decision tree algorithms, this issue arises because the tree can grow excessively deep, making splits that overly conform to the training data, including its noise and anomalies. This tendency of decision trees to create highly specific branches leads to a model that is overly complex and lacks generalization. To mitigate overfitting, techniques such as tree pruning, limiting the tree's maximum depth, requiring a minimum number of samples per leaf, and using ensemble methods like bagging and boosting are employed to reduce complexity and enhance the model's generalization ability.

4 EXPERIMENT 2

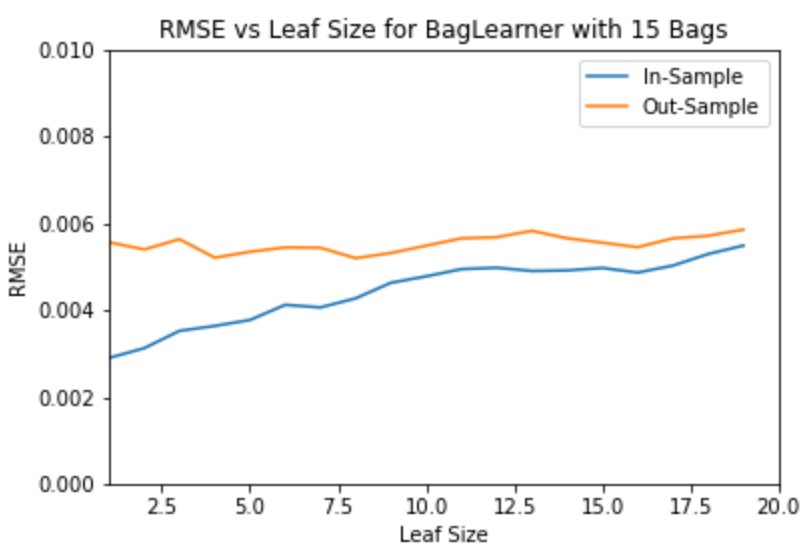

Figure 2. RMSE versus change in Leaf Size for In and Out-of-sample for the BagLearner containing 15 bags each with Independent DTLearner instances.

Can bagging reduce or eliminate overfitting with respect to leaf size?

Bagging is shown to decrease overfitting with respect to leaf size as shown in Figure 2. RMSE values reduced across all leaf sizes for In and Out-of-sample with a In-sample RMSE decrease of 0.002 for Out-sample and 0.001 decrease for In-sample for a leaf size of 1. Greater performance with respect to decrease in overfitting with Bagging could be explained by this experiment due to each instance of the learner for the bags being the same DTlearner and not allowing a more robust ensemble of different learners for better generalization. Overfitting was not eliminated using Bagging in this case but reduced.

5 EXPERIMENT 3

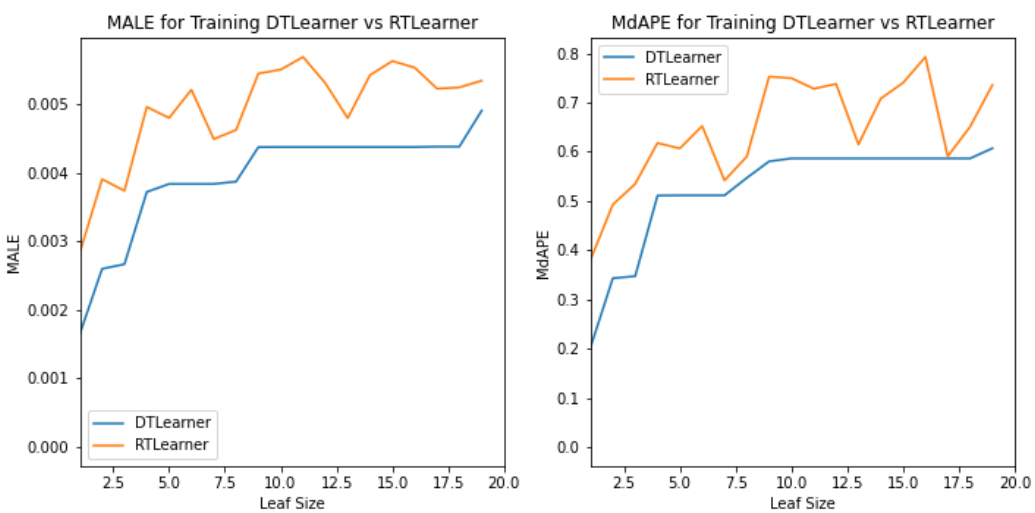

Figure 3. Mean Absolute Logarithmic Error(MALE) and Median Absolute Percentage Error(MdAPE) for DTLearner and RTLearner versus Leaf Size.

Which ways is one method better than the other and which learner had better performance in this experiment and likely to be better in general?

Figure 3 depicts the comparison of performance metrics MALE and MdAPE for the DTLearner and RTLearner. MALE is the average of the absolute differences between the logarithms of the predicted and actual values with this metric used to stabilize variance and reduce impact of outliers. The MdAPE metric measure the median of the absolute percentage errors of predicted and actual label values. The DTLearner performed better in the MALE experiment with lower MALE metric values across all leaf sizes. The same can be seen for the MdAPE experiment. The jagged fluctuations in both metric values for the RTLearner and lower performance overall compared to DTLearner could be explained by either the inherent random feature selection, less consistent splits, sampling variability, and alternation of over and underfitting with leaf size change. RTLearner could prove to be more beneficial in ensemble techniques aside from being a standalone model where the random trees contribute to overall model performance and smoother performance curves thus one is not always likely to be superior to another and depends on the context of how the learner is used.

6 SUMMARY

Overall, the experiments demonstrated the strengths and weaknesses of each learner using the financial stock market Istanbul dataset. The DTLearner is more effective as a standalone model in this case due to its optimized and consistent splits, resulting in lower error rates and smoother performance. The BagLearner effectively reduces overfitting(although not substantially or fully eliminating due to using the same type of model) by aggregating multiple models, leading to more robust predictions. The RTLearner, while less reliable on its own, can provide valuable diversity in ensemble methods, enhancing generalization. Thus, the choice of learner depends on the specific use case, data characteristics, and whether ensemble methods are employed. Further experimentation on other machine learning model techniques on the same dataset and same train/test split seed can provide further insights into optimal directions of predictions regarding stock market data in the financial domain, specifically on the Istanbul dataset of stock returns for EM.

7 RESOURCES

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106. https://doi.org/10.1007/bf00116251